

Relatório de Implementação do Matchmaking System

Gustavo Silva dos Santos
João Pedro Yoshida de Oliveira

1 Descrição Geral

O projeto desenvolvido implementa um sistema simples de *matchmaking* para partidas de jogos. O **Matchmaking** possui como objetivo armazenar jogadores em uma fila de espera, ordenar esses jogadores de acordo com seu nível (pontos) e formar grupos válidos para partidas online.

Cada jogador possui quatro atributos principais: `id`, `name`, `score` e `timestamp`. O `score` representa o nível do jogador, enquanto o `timestamp` representa sua ordem de chegada na fila. O `timestamp` é utilizado como critério de desempate quando dois jogadores possuem o mesmo `score`.

A estrutura principal do sistema é a classe **Matchmaking**, responsável por armazenar os jogadores em um array de tamanho fixo e realizar todas as operações do sistema.

2 Implementação dos Algoritmos de Ordenação

Foram implementados os dois algoritmos de ordenação: **Insertion Sort** e **Merge Sort** como métodos da classe **Matchmaking**.

O **Insertion Sort** foi implementado percorrendo o array de jogadores da esquerda para a direita. A cada iteração, o jogador atual é inserido na posição correta da parte já ordenada do array. Durante o processo, os elementos maiores são deslocados uma posição para a direita até que seja encontrada a posição correta do jogador atual. A ordenação utiliza dois critérios: menor `score` e, em caso de empate de pontos, menor `timestamp`. Essa implementação possui custo $O(n)$, pois, no pior caso, cada elemento pode precisar deslocar todos os elementos anteriores.

Já o **Merge Sort** foi implementado utilizando divisão e conquista. O array é dividido recursivamente em duas metades e cada uma das metades também é dividida em outras duas, e assim por diante até que cada subarray possua tamanho 1. Em seguida, os subarrays são intercalados em ordem crescente por `score`, formando um array ordenado completo dos jogadores.

3 Uso da Recursão no Merge Sort

A recursão foi utilizada para dividir o problema de ordenação em problemas menores. A função **MergeSort** recebe um array e seu tamanho. Caso o tamanho seja menor ou igual a 1, a função retorna imediatamente, pois arrays desse tamanho já estão ordenados.

Caso contrário, o array é dividido em duas metades (esquerda e direita). Cada metade é ordenada recursivamente utilizando novas chamadas da própria função **MergeSort**. Após as chamadas recursivas, os dois arrays ordenados são intercalados através do processo de *merge*.

Durante a intercalação, o algoritmo compara os menores elementos disponíveis de cada metade e insere no array final aquele que deve vir primeiro na ordenação. Esse processo continua até que todos os elementos sejam copiados para o array final ordenado.

A utilização da recursão simplifica significativamente a implementação do algoritmo, além de permitir um custo assintótico melhor do que o **Insertion Sort**.

4 Formação dos Grupos

A ordenação dos jogadores foi utilizada para simplificar a formação de grupos válidos. Após ordenar os jogadores por score, o sistema passa a considerar apenas grupos consecutivos no array. Isso é possível porque, após a ordenação, jogadores com scores semelhantes ficam próximos uns dos outros. O sistema percorre o array verificando janelas consecutivas de tamanho `groupSize`. Para cada grupo candidato, basta comparar o **maior score** com o **menor score**. Caso essa diferença seja menor ou igual ao valor de `delta` (diferença máxima aceitável), o grupo é considerado válido.

Essa abordagem reduz bastante a complexidade da busca, pois elimina a necessidade de testar combinações arbitrárias de jogadores.

5 Custo Computacional

A operação de inserção possui custo $O(1)$, pois o jogador é sempre adicionado ao final do array, enquanto a remoção possui custo $O(n)$, pois, no pior caso, é necessário percorrer todo o array para encontrar o jogador desejado (e após encontrá-lo, os elementos posteriores precisam ser deslocados uma posição para a esquerda para preencher o espaço removido).

Sobre os algoritmos, o **Insertion Sort** possui custo $O(n^2)$ no pior caso, porque cada novo elemento inserido pode precisar deslocar todos os elementos anteriores até encontrar sua posição correta no array ordenado. Já o **Merge Sort** possui custo $O(n \log n)$, já que o algoritmo divide recursivamente o array em partes menores e realiza a intercalação dos subarrays ordenados em tempo linear a cada nível da recursão. Além desses, a formação de grupos possui custo $O(n)$ após a ordenação dos jogadores, pois (como o array já está ordenado por score) basta percorrer grupos consecutivos de tamanho fixo e verificar a diferença entre o menor e o maior score de cada grupo candidato.

Por fim, a operação de recuperação dos jogadores através do método `getWaitingPlayers` possui custo $O(n)$, pois é necessário copiar todos os jogadores armazenados para um novo array dinamicamente alocado.

6 Comparação Experimental

Foram realizados testes simples para comparar o desempenho dos algoritmos *Insertion Sort* e *Merge Sort*. Os testes foram executados utilizando conjuntos equivalentes de jogadores gerados artificialmente com scores aleatórios.

Os tempos foram medidos utilizando a biblioteca `chrono` do C++.

Quantidade de jogadores	Insertion Sort	Merge Sort
100	0.1 ms	0.07 ms
500	2.2 ms	0.5 ms
1000	15.9 ms	1.3 ms
5000	241 ms	7.9 ms
10000	1406 ms	18 ms

Os resultados observados estão de acordo com a complexidade esperada dos algoritmos. O *Insertion Sort* apresentou crescimento quadrático, tornando-se rapidamente mais lento conforme o tamanho da entrada aumentava.

Por outro lado, o *Merge Sort* apresentou crescimento significativamente menor, mantendo tempos relativamente baixos mesmo para entradas maiores.

Isso ocorre porque o *Merge Sort* possui complexidade $O(n \log n)$, enquanto o *Insertion Sort* possui complexidade $O(n^2)$.

7 Conclusão

A implementação realizada se mostrou bem adaptada às necessidades de controle e ordenação de jogadores em um sistema simples de *matchmaking*. A escolha de arrays estáticos para gerenciar as filas permitiu uma implementação organizada e relativamente eficiente para o trabalho.

Analisando os resultados dos testes realizados, o *Insertion Sort* apresentou implementação mais simples, porém desempenho inferior para grandes quantidades de jogadores. Já o *Merge Sort* exigiu uma implementação mais complexa devido à recursão e separação/fusão das partes das filas, mas apresentou desempenho muito superior.

Além disso, a utilização da ordenação facilitou significativamente a formação de grupos válidos, permitindo uma busca eficiente por jogadores com níveis semelhantes.